

[알고리즘 과제] 허프만 코드를 이용한 알고리즘 - 3주차과제 보고서

- 학번: 2371214
- 이름: 이지형
- 트랙: 모바일소프트웨어&빅데이터 트랙

1. 과제 개요

최소 힙 알고리즘을 활용하여 문자 출현빈도수에 허프만 코드를 구축하고, 이를 통한 인코딩 및 디코딩 함수를 구현하였습니다.

기존 교재에 있는 허프만코드에서 확장해서 과제의 요구사항(동적 메모리공간 할당, 식별자부여, 텍스트 파일 연동, 비트 변환 알고리즘)을 반영하기 위해 과제를 시행하였습니다.

2. 주요 구현 함수

교재의 기본 알고리즘 구조(교재352페이지)를 바탕으로, 과제의 요구사항을 해결하기 위해 추가한 핵심 함수 목록입니다.

2.1. huffman_tree() — 내부 노드 이름 자동 부여

구현 방식: 교재에 작성되어 있는 함수는 단순히 노드를 합치는 데 집중되어 있어서, h_count라는 카운터 변수를 두고 트리가 만들어질 때마다 H-1, H-2 형태의 문자열을 생성해 노드 name에 넣어주도록했습니다.

2.2. fixed_bit() — 고정 비트 길이 계산 함수

작성 이유: 검색을 해보니 math.h의 log2나 ceil 같은 수학 함수를 사용해야 해서, 추가적으로 함수를 사용하면 더 복잡할것같아 해당 함수들을 사용하지 않고 간단하게 비트 수를 구하고 싶었습니다.

구현 방식: 문자 개수 N 에 대해 $2^k \geq N$ 이 되는 지점까지 2를 계속 곱해주는 while문을 사용해 비트 수를 구할수 있도록 작성했습니다.

2.3. search_encode() & encode_string() — 문자열 인코딩 함수

작성 이유: 교재 print_codes 함수는 화면에 출력만 해주기 때문에, 입력받은 문장을 실제로 0과 1 비트로 바꾸려기위해 별도의 함수를 작성했습니다.

구현 방식:

search_encode(): 루트부터 시작해서 왼쪽으로 가면 '0', 오른쪽으로 가면 '1'을 붙여가며 트리를 재귀적으로 탐색하고, 내가 찾는 문자를 만나면 누

적된 비트열을 완성해서 돌아옵니다.

2.4. decode_string() — 비트열 디코딩 함수

작성 이유: 0과 1의 비트를 다시 원래의 글자로 복원하기 위해 작성했습니다.

구현 방식: 비트열에서 '0'이 나오면 트리의 왼쪽 자식으로, '1'이 나오면 오른쪽 자식으로 이동합니다. 그러다 끝 노드(is_leaf함수를 사용)에 도착하면 그 노드의 글자를 결과에 적어두고, 다시 루트 노드로 돌아가 다음 비트를 읽어서 해석하도록 만들었습니다.

3. 고정 비트 vs 허프만 비트 압축률 비교

1. 고정비트로 표현할시? (4-bit Code):

- $N = 16$ 이므로 문자에 4비트 할당 → 총 1,112 비트가 필요

2. 허프만 가변 비트 표현 시 (Huffman Code):

- 빈도수가 높은 문자일수록 짧은 비트 할당 → 총 1,014 bits 필요

3. 결과 분석: 실습에 사용한 메모장 문서를 기준으로 허프만 프로그램을 실행할시 고정 비트 방식 대비 계산상 8.8%의 데이터 절감 효과가 있습니다.

4. 결론 및 종합 소감.

단순히 교재에 있는 허프만 트리를 형성하고 출력하는 데서 그치지 않고, 트리를 활용한 과제의 다양한 조건에서의 코드를 구조화하고 함수를 만들면서 탐색 알고리즘과 포인터 연산에 대한 이해도를 크게 높일 수 있는 계기가 된 것 같습니다. 특히 과제를 수행하려면 이전에 있던 전위순회코드,우선순위큐,허프만코드등등 다양한 코드들을 인지해야해서 다시 모든 코드들을 교재를 통해 복습하는 시간이 오래걸려 힘들긴 했지만, 그래도 과제를 마치니 복습한 내용들이 도움이 된것같아서 보람있습니다.

알고리즘을 직접구현하며 포인터 표현에 있어 막히는 부분이 많았는데, 제가 코드를 작성하면서 부족한부분들이 어떤것들인지 다시 한번 확인할수 있는 시간이였습니다.

